

UNIVERSIDAD PERUANA UNIÓN
FACULTAD DE INGENIERÍA Y ARQUITECTURA
E.P. DE INGENIERÍA DE SISTEMAS



**LABORATORIO: REGISTRO TRANSACCIONAL DE
PEDIDOS CON CABECERA-DETALLE**

CURSO:
Lenguaje de programación II

DOCENTE:
Ing. Benjamín David Reyna Barreto

ESTUDIANTE:
Mishelle Esmeralda Osorio Huashuayo

LIMA, AGOSTO 2026

Repositorio GitHub: <https://github.com/mishi1307/pharmaSoft.git>

1. DESCRIPCIÓN DE LA SOLUCIÓN Y ARQUITECTURA

La presente actividad implementa una operación REST cabecera-detalle para el registro de ventas/pedidos de forma transaccional. La solución garantiza la **atomicidad**, el cálculo automático de subtotales/totales y el control estricto de existencias (stock).

Estructura de Paquetes

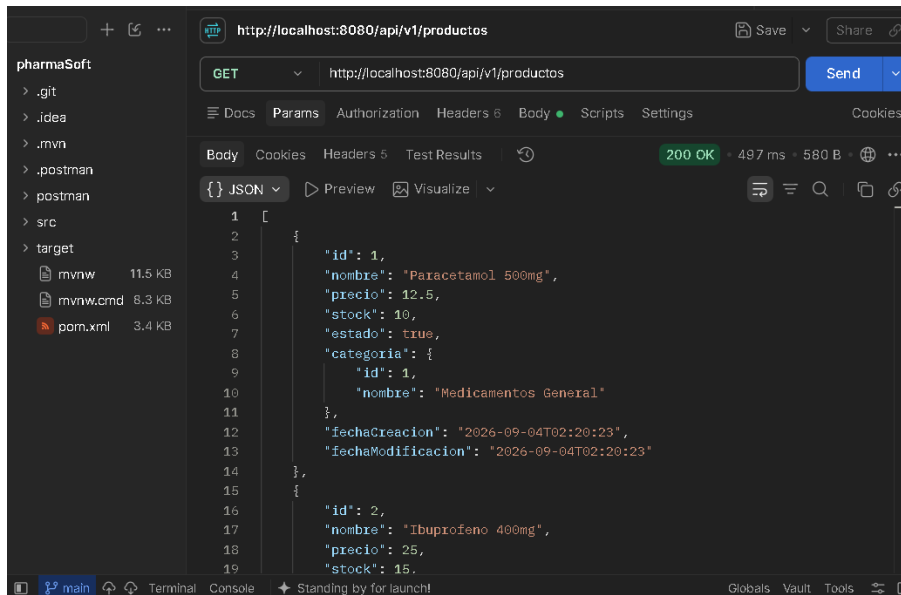
- **entity:** Venta, DetalleVenta, Producto, Cliente, Categoria.
- **dto:** CategoriaDTO, CategoriaRequestDTO, CategoriaResponseDTO, CategoriaResumenDTO, ClienteRequestDTO, ClienteResponseDTO, DetalleVentaRequestDTO, DetalleVentaResponseDTO, ErrorResponseDTO, ProductoDTO, ProductoRequestDTO, ProductoResponseDTO, VentaRequestDTO, VentaResponseDTO.
- **repository:** CategoriaRepository, ClienteRepository, ProductoRepository, VentaRepository, DetalleVentaRepository.
- **service:** se divide en tres subcarpetas Service(CategoriaService, ClienteService, ProductoService, VentaService), Generic(CrudService)e Impl(CategoriaServiceImpl, VentaServiceImpl, ClienteServiceImpl, ProductoServiceImpl).
- **controller:** CategoriaController, ClienteController, HealthController y VentaController (endpoint POST /api/v1/ventas), ProductoController.
- **exception:** ReglaNegocioException, StockInsuficienteException, RecursoNoEncontradoException, GlobalExceptionHandler.
- **Enums:** EstadoVenta.

2. EVIDENCIAS DE EJECUCIÓN Y PRUEBAS FUNCIONALES

2.1. Escenario 1: Operación Exitosa (Commit)

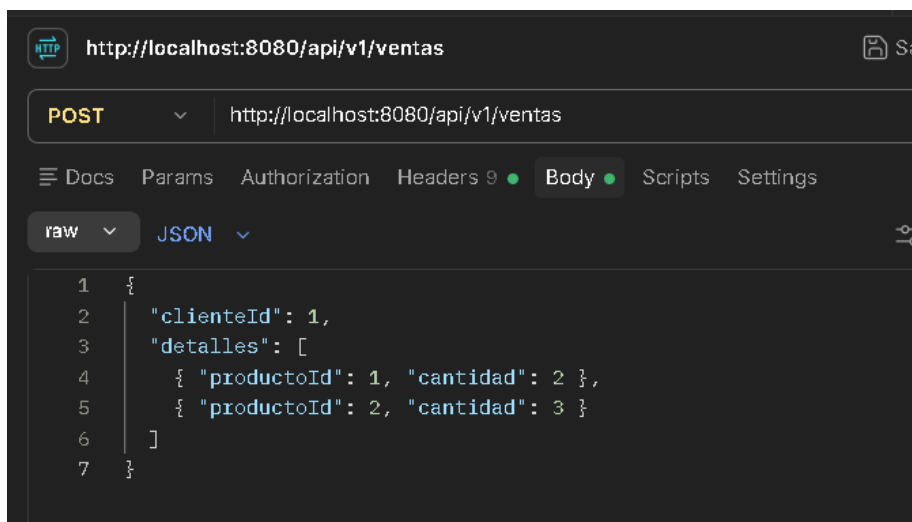
Paso 1: Stock previo en la Base de Datos

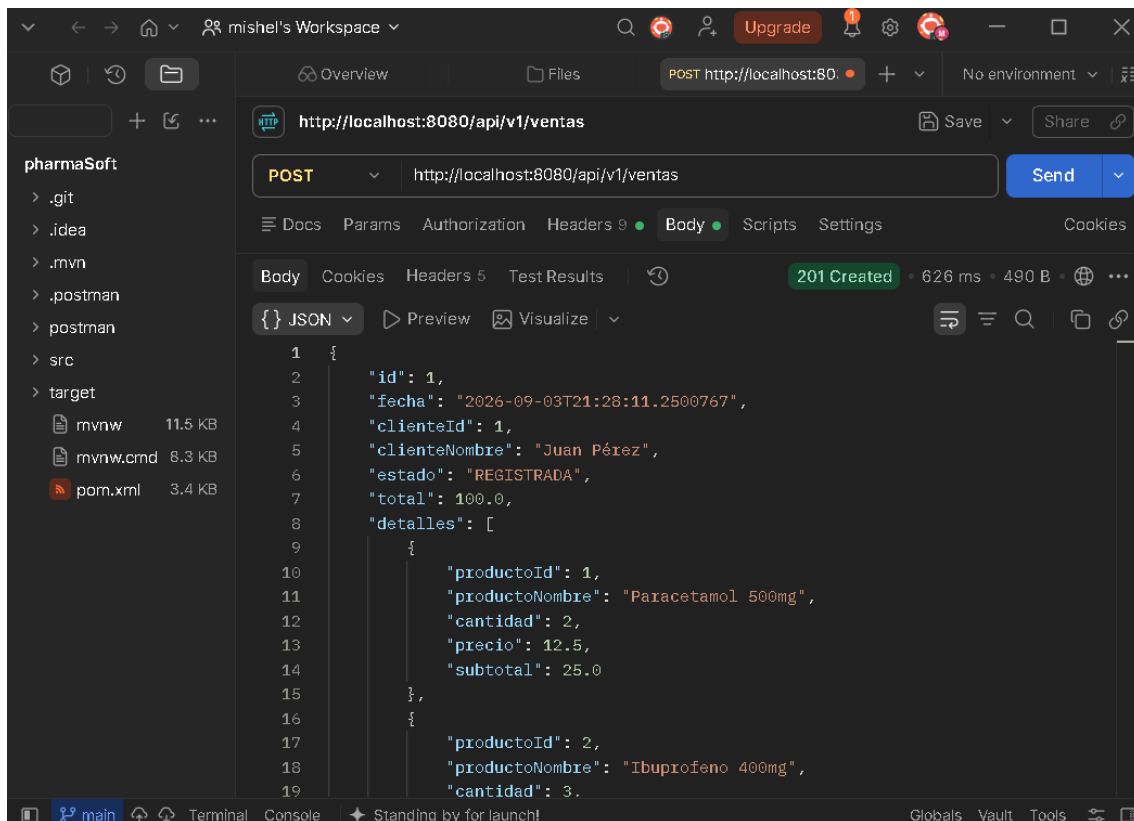
- **Método/URL:** GET <http://localhost:8080/api/v1/productos>
- **Descripción:** Se verifica el estado inicial de las existencias de los productos antes de realizar la compra.



Paso 2: Registro del Pedido

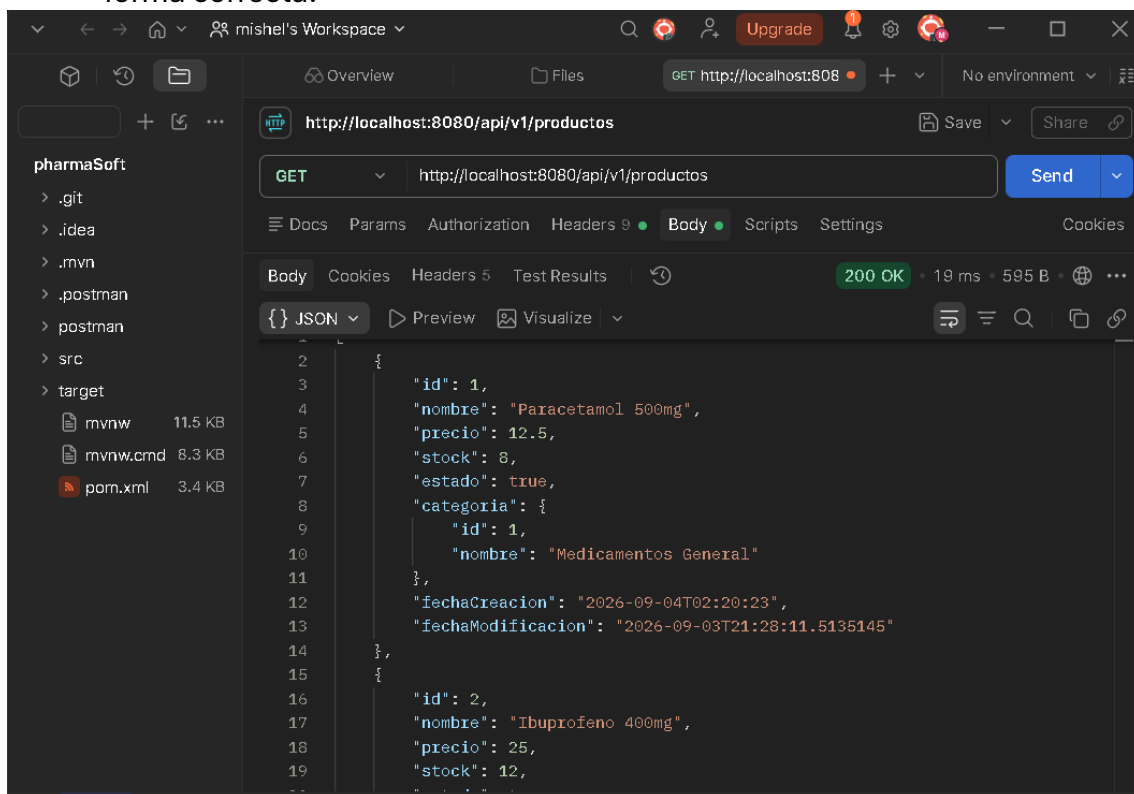
- **Método/URL:** POST `http://localhost:8080/api/v1/ventas`
- **JSON Enviado (Request Body):**





Paso 3: Verificación del Stock Resultante

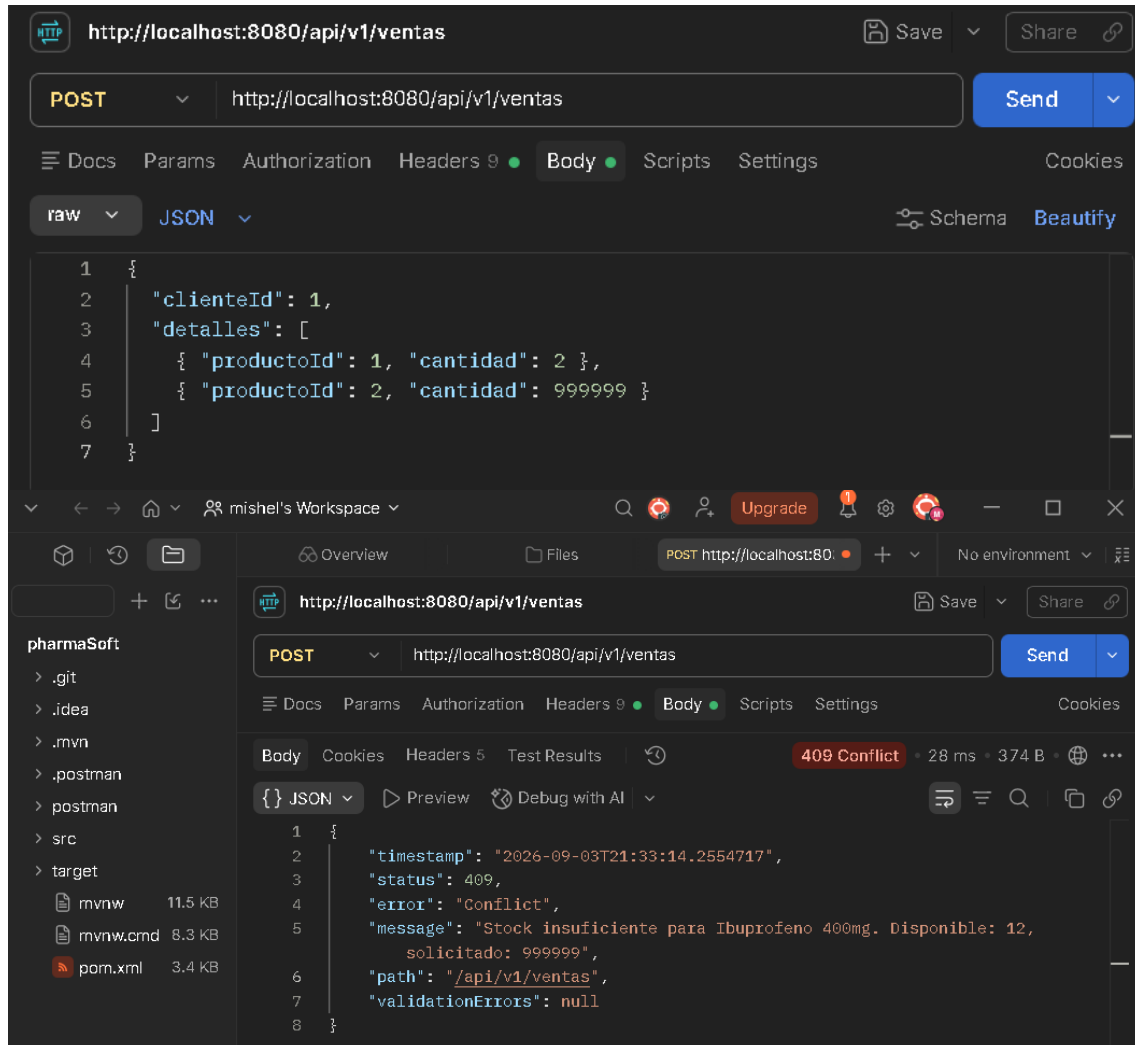
- **Método/URL:** GET `http://localhost:8080/api/v1/productos`
- **Descripción:** Comprobación de que la transacción redujo el inventario de forma correcta.



2.2. Escenario 2: Stock Insuficiente y Demostración de Rollback

Paso 1: Intento de registro con exceso de cantidad

- **Método/URL:** POST <http://localhost:8080/api/v1/ventas>
- **Descripción:** Se envía un pedido donde el primer producto tiene una cantidad válida pero el segundo supera el stock disponible para forzar una excepción en la mitad de la iteración.
- **JSON Enviado (Request Body):**



Paso 2: Comprobación de Rollback en Base de Datos

- **Método/URL:** GET <http://localhost:8080/api/v1/productos>
- **Descripción:** Se comprueba que, gracias a @Transactional, el Producto 1 **NO sufrió ninguna reducción parcial** de stock a pesar de ser procesado antes de que fallara el Producto 2.

mishe!l's Workspace

Overview Files GET http://localhost:8080

http://localhost:8080/api/v1/productos Save Share

GET http://localhost:8080/api/v1/productos Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Body Cookies Headers 5 Test Results 200 OK • 12 ms • 595 B •

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "nombre": "Paracetamol 500mg",
5     "precio": 12.5,
6     "stock": 8,
7     "estado": true,
8     "categoria": {
9       "id": 1,
10      "nombre": "Medicamentos General"
11    },
12     "fechaCreacion": "2026-09-04T02:20:23",
13     "fechaModificacion": "2026-09-03T21:28:11.5135145"
14  },
15  {
16    "id": 2,
17    "nombre": "Ibuprofeno 400mg",
```